



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Asignatura:
Sistemas Operativos

Asignación:
Tarea #5: LAB 0, calentamiento al Módulo III con Redis

Profesor:
Prof. Isacio Tamayo

Estudiante:
James Tack

Grupo:
1IL143

Fecha de entrega:
20 de octubre de 2026

2do Semestre 2026

De qué se trata la tarea

Esta tarea es el LAB 0 del Módulo III: un calentamiento de una hora con Redis como hilo conductor. En una sola sesión se tocan los tres temas del módulo: los descriptores de archivo y el VFS, el page cache con fsync, y dos errores reales (EMFILE y ENOSPC). Lo que se entrega es la tabla de cierre completa y las capturas de la terminal de los pasos 1, 4 y 5. Pongo también las de los pasos 2, 3 y 6 porque la tabla se apoya en ellas.

Todo se ejecutó dentro de la máquina virtual del curso (vm-sistemas-operativos, Ubuntu 18.04, kernel 4.15), sin permisos de administrador, salvo el paso 6a, que la guía marca como opcional y pide sudo. La VM no traía Redis, así que instalé el paquete real (redis-server 4.0.9) con apt y lo dejé detenido como servicio del sistema. El script redis-lab0.sh lanza una instancia efímera en el puerto 7777, por lo que en el paso 5 se ven los descriptores de un Redis de verdad y no los de un shell. Cada captura sale de una ventana de Windows Terminal conectada por ssh a la VM, con mi nombre en el prompt.

Preparación del entorno

Paso 1. El kernel detrás de Redis

Copié redis-lab0.sh y baseline.sh a ~/laboratorios/tarea5/scripts en la VM y ejecuté el setup. Como redis-server está instalado, el script lo lanza en el puerto 7777 con el directorio /tmp/redis-lab0, appendonly activado y el log en redis.log. También deja un dump.rdb simulado de 1 MiB y un appendonly.aof vacío para los pasos siguientes.

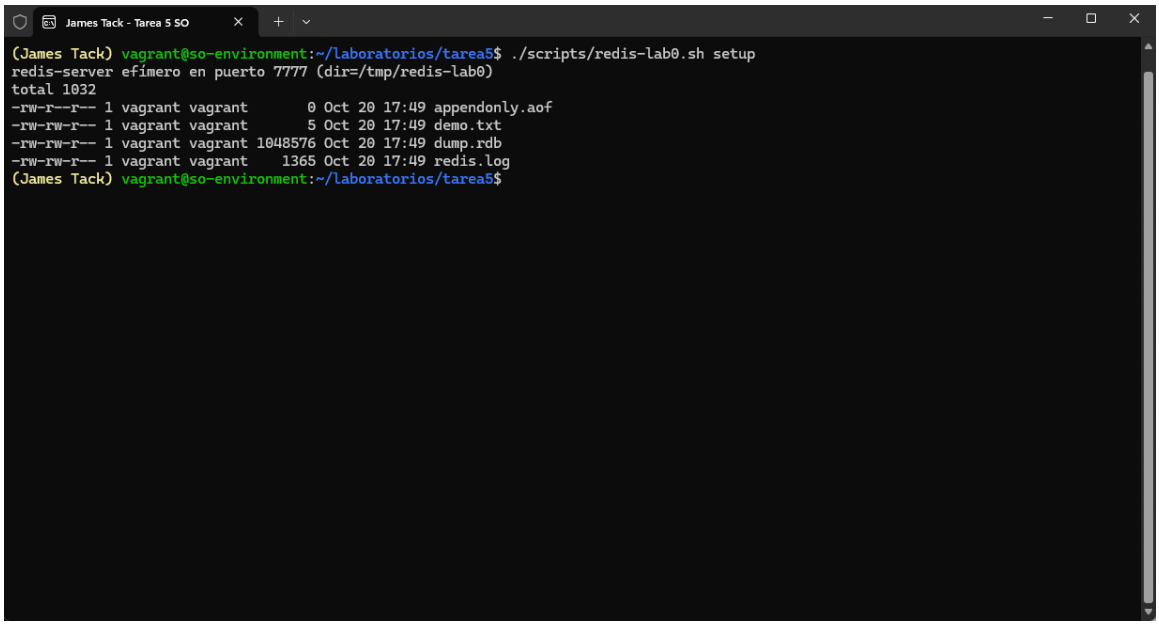


Figura 1. Paso 1: el setup lanza redis-server en el puerto 7777 y deja dump.rdb, appendonly.aof y demo.txt en /tmp/redis-lab0.

Después guardé el PID del servicio en $SVC con pgrep y leí su cmdline y su status en /proc. El proceso se llama redis-server, escucha en *:7777, está dormido (S), su padre es el PID 1 porque se lanzó con daemonize, ocupa 3848 kB de memoria residente y tiene 4 hilos. El kernel es el 4.15.0-213-generic y la VM llevaba 12 minutos encendida.

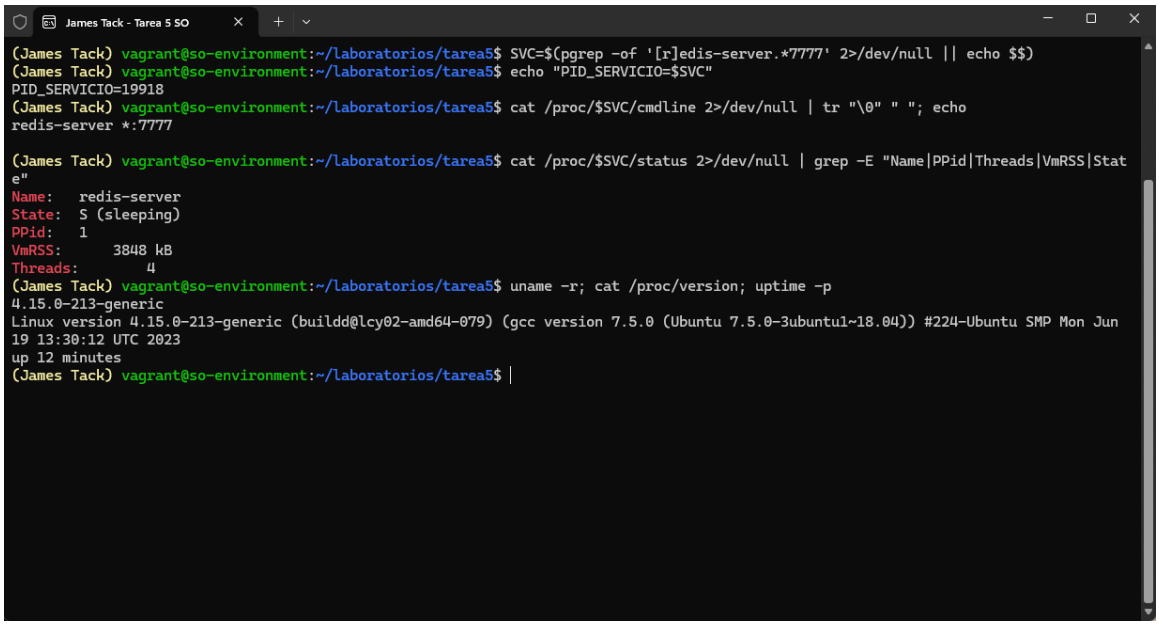


Figura 2. Paso 1: PID 19918, cmdline y status de redis-server leídos desde /proc; kernel 4.15.0-213-generic.

Lo que muestra. /proc no es un archivo: cada lectura pide al kernel los datos de la task_struct del proceso en ese momento. Por eso el PPid y el estado salen sin que Redis haga nada.

Pasos 2 y 3. Memoria e inodos

Paso 2. La memoria que el SO presta a los datos

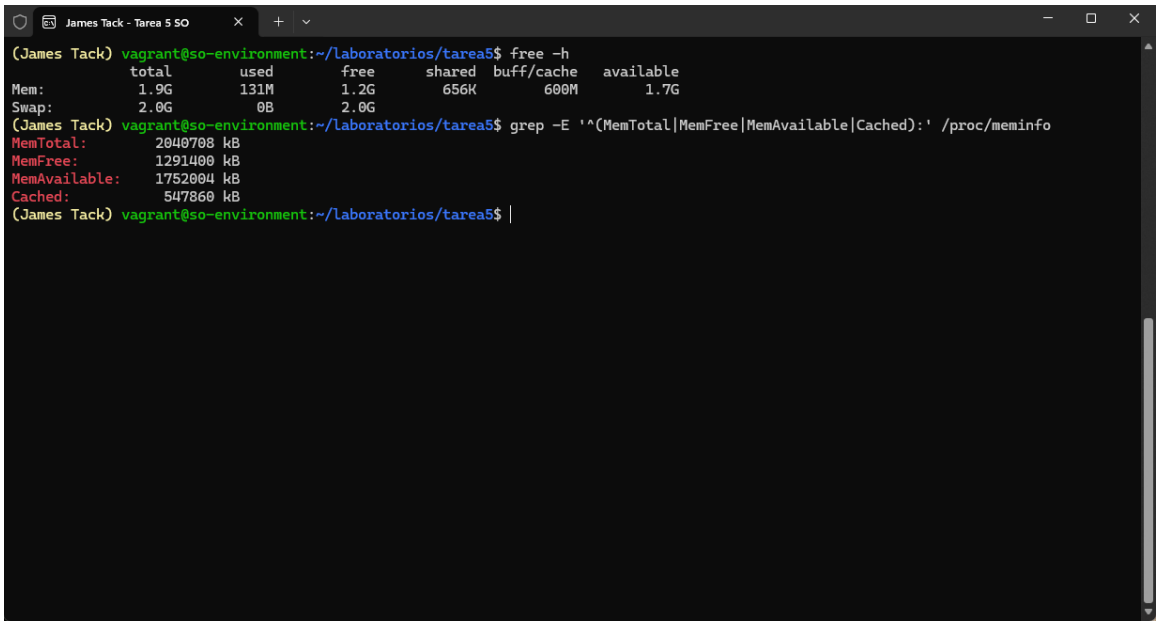


Figura 3. Paso 2: free -h y los campos de /proc/meminfo; buff/cache 600M y MemAvailable 1752004 kB.

free -h marca buff/cache de 600M, y /proc/meminfo da MemTotal 2040708 kB, MemFree 1291400 kB, MemAvailable 1752004 kB y Cached 547860 kB. Esa memoria no está perdida: es page cache que el kernel presta a los archivos (por ejemplo, el dump.rdb que Redis lee al arrancar) y la devuelve en cuanto un proceso la pide. Por eso MemAvailable (1.7 GB) es mayor que MemFree (1.2 GB): al libre se le suma la parte del cache que se puede reclamar.

Qué muestra	Valor en mi máquina	Respuesta (pensando en Redis)
buff/cache	600M (Cached = 547860 kB)	Es memoria prestada al dataset y a los archivos RDB/AOF, no "usada" por ningún proceso; el kernel la recupera cuando hace falta.
MemAvailable	1752004 kB (1.7G), frente a MemFree de 1291400 kB	Available es mayor que free porque suma el page cache reclamable.

Paso 3. Inodos: dónde guarda Redis lo persistente

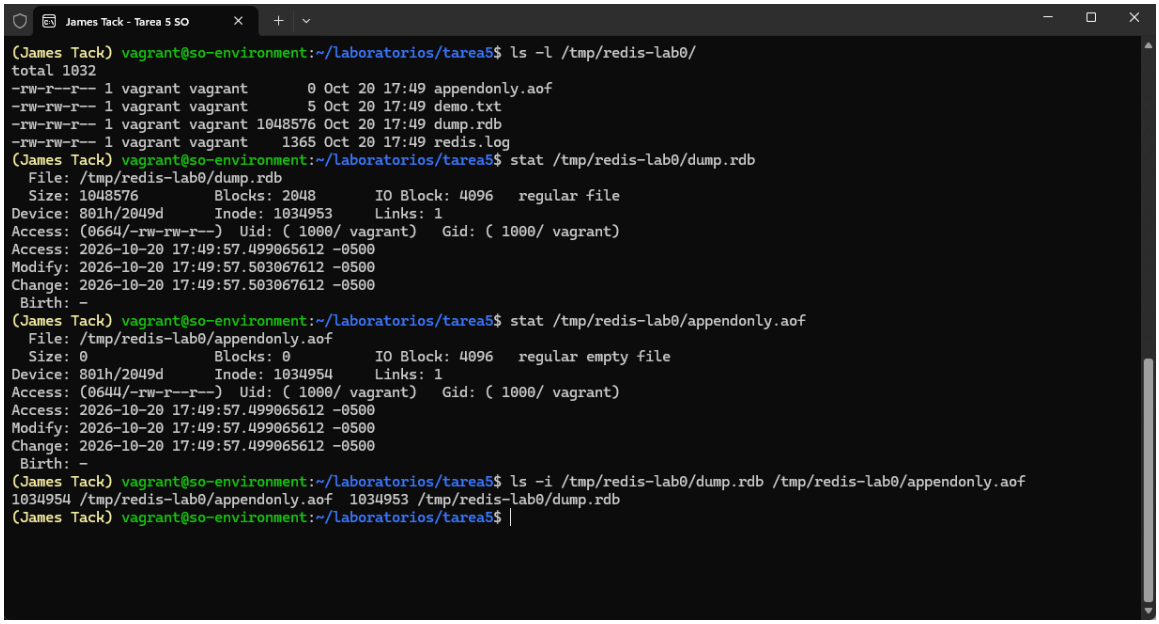


Figura 4. Paso 3: ls -l, stat e ls -i de dump.rdb y appendonly.aof.

dump.rdb tiene el inodo 1034953, 1 048 576 bytes y 2048 bloques de 512 bytes; appendonly.aof tiene el inodo 1034954 y 0 bytes, porque todavía no se ha ejecutado ninguna escritura sobre Redis. Cada archivo tiene su propio inodo, y el inodo es quien guarda el tamaño, los permisos, los tiempos y los bloques; el nombre vive en el directorio.

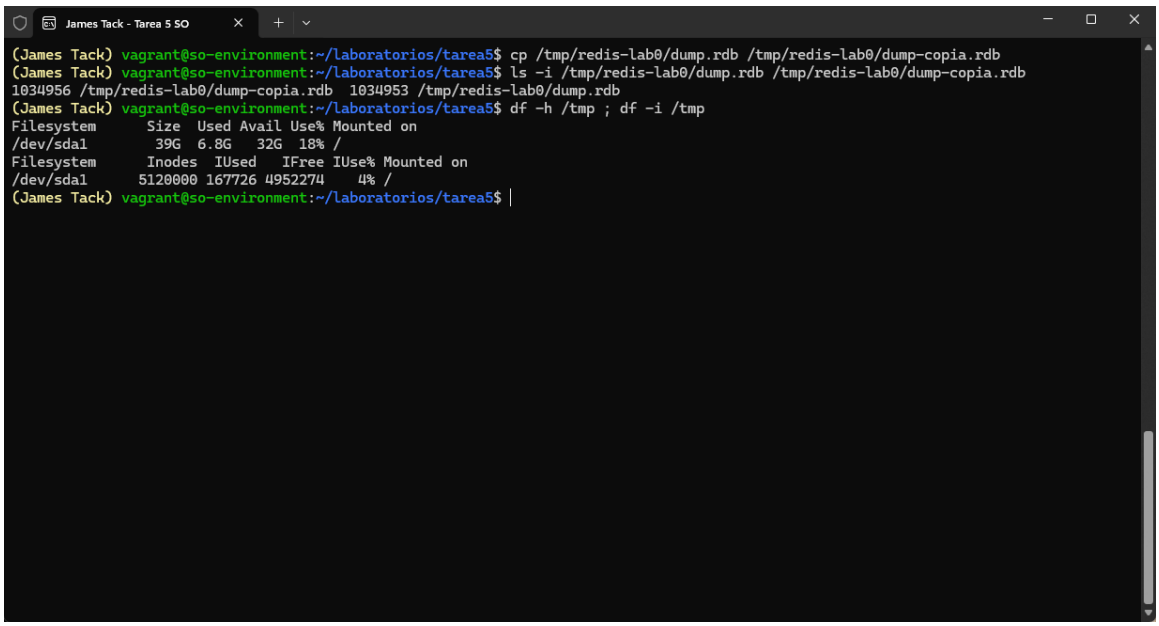


Figura 5. Paso 3: la copia de dump.rdb recibe otro inodo; df -h usa 18 % del espacio y df -i usa 4 % de los inodos.

Al copiar con cp, dump-copia.rdb recibió el inodo 1034956, distinto del 1034953 del original. Son dos archivos independientes con el mismo contenido; solo un enlace duro (ln) compartiría el inodo.

En /tmp, df -h marca 39G de tamaño con 18 % usado y df -i marca 5 120 000 inodos con 4 % usado. Son dos límites diferentes: se puede agotar el espacio con inodos libres, o los inodos con espacio libre.

Paso 4. Page cache con un archivo tipo RDB

Creé un archivo de 300 MiB de datos aleatorios (rdb.sim) para hacer de dataset, lo expulsé del page cache solo a él con posix_fadvise(POSIX_FADV_DONTNEED) y lo leí dos veces con dd midiendo el tiempo. Entre una y otra miré Cached, Dirty y Writeback.

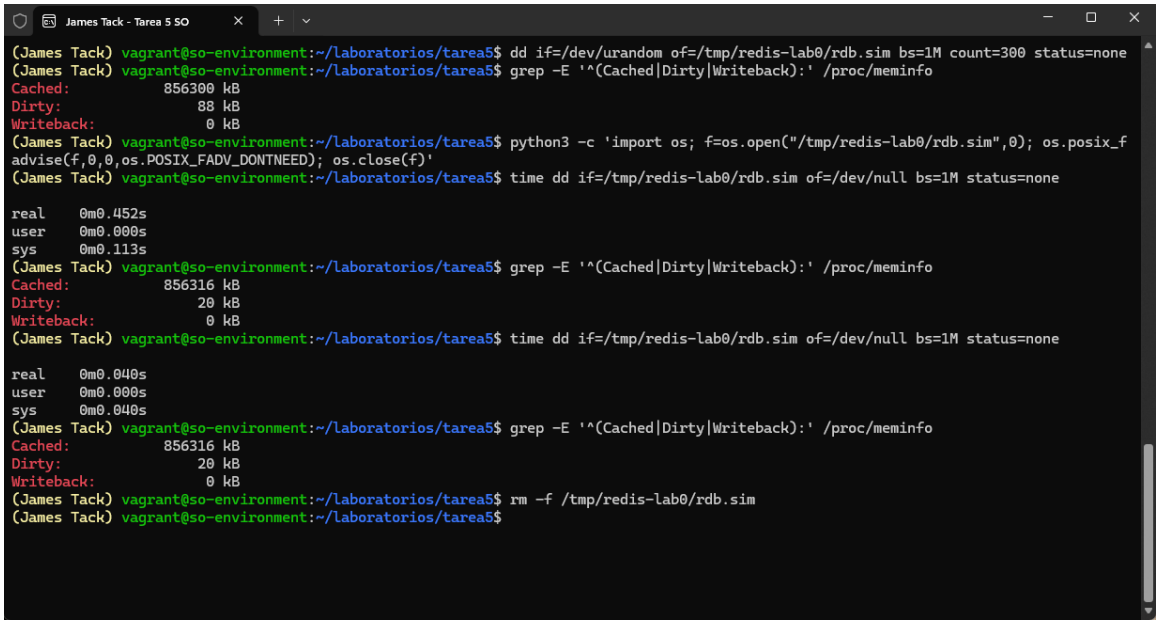


Figura 6. Paso 4: la primera lectura tarda 0.452 s (desde disco) y la segunda 0.040 s (desde el page cache).

Resultado. La primera lectura tardó 0.452 s de tiempo real y la segunda 0.040 s, unas 11 veces menos. Cached pasó de 856300 kB a 856316 kB al terminar la primera lectura: el archivo volvió a entrar en el page cache y la segunda vez no hizo falta ir al disco. Dirty se mantuvo en 88 kB y 20 kB, es decir, los 300 MiB ya habían bajado a disco. Esto es lo que pasa cuando Redis reinicia: si dump.rdb o appendonly.aof siguen en el page cache, la carga y el BGREWRITEAOF son mucho más rápidos. Estos dos tiempos son mi primera evidencia para el LAB 2.

Paso 5. Descriptores de Redis y appendfsync

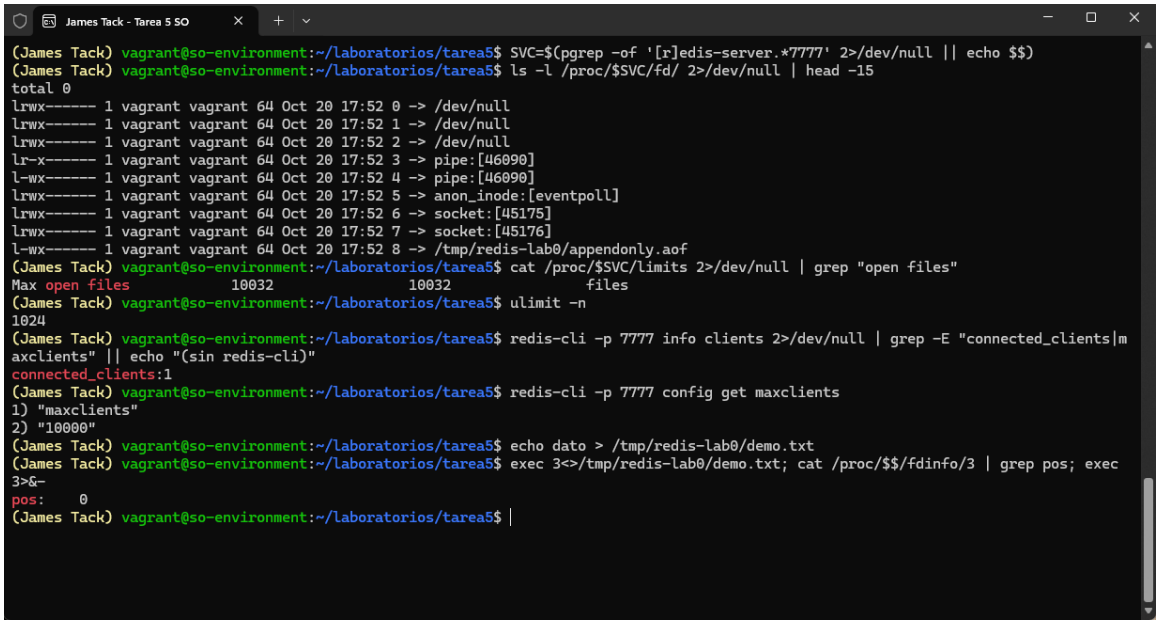


Figura 7. Paso 5: /proc/<redis>/fd muestra 9 descriptores; el límite del proceso es 10032, el del shell es 1024 y el pos del descriptor 3 es 0.

Esta es la parte que más cambia por tener un Redis real. En /proc/19918/fd aparecen los descriptores 0, 1 y 2 apuntando a /dev/null (Redis es un demonio), el 3 y el 4 como los dos extremos de un pipe, el 5 como anon_inode:[eventpoll] (el bucle de eventos), el 6 y el 7 como sockets (el que escucha en el puerto 7777 y otro más) y el 8 como /tmp/redis-lab0/appendonly.aof. Es decir, un socket de escucha y el archivo AOF son entradas de la misma tabla de descriptores, que es la idea del VFS.

Límites. Max open files del proceso es 10032, mientras que el shell tiene 1024 (ulimit -n). Redis sube su propio límite a maxclients más 32 descriptores de reserva para el AOF, el log y el bucle de eventos: maxclients es 10000, y 10000 + 32 = 10032. Si el límite del proceso no alcanzara, Redis tendría que bajar maxclients al arrancar y escribirlo en el log. connected_clients vale 1: es el propio redis-cli que hace la consulta. Cada cliente nuevo gasta un socket y por lo tanto un descriptor.

Offset en la open file description. Con exec 3<> abrí demo.txt y leí /proc/$$/fdinfo/3: el pos es 0 porque el descriptor se acaba de abrir. El desplazamiento vive en la open file description del kernel, no en el archivo ni en el número de descriptor.

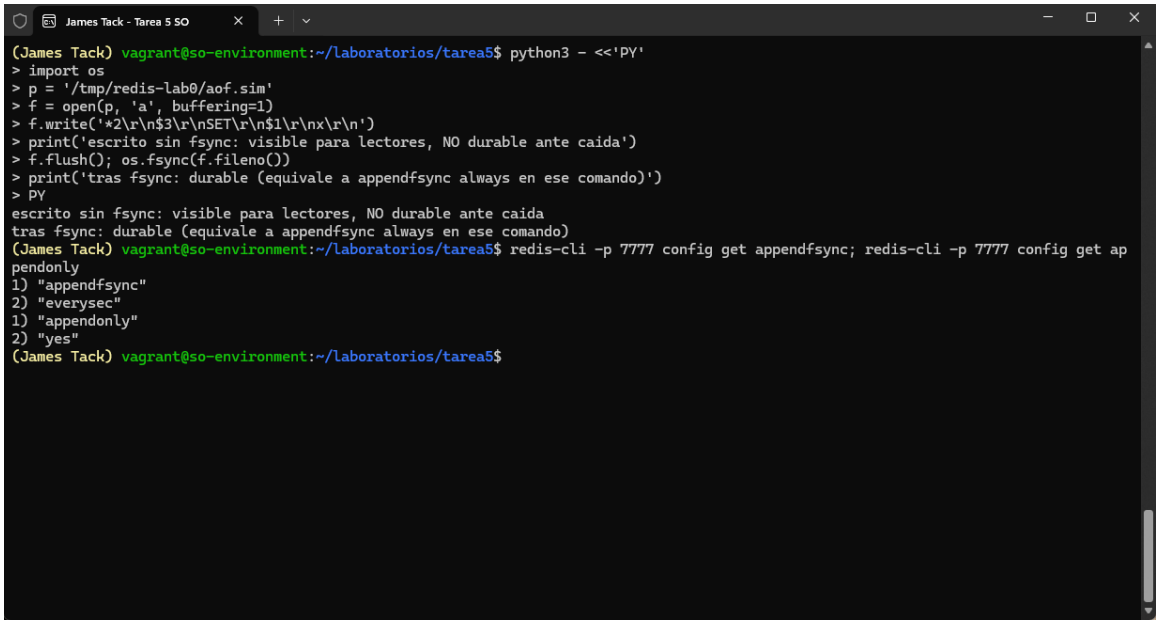


Figura 8. Paso 5: demo de fsync con Python y valores de appendfsync (everysec) y appendonly (yes) en la instancia real.

appendfsync. La demo con Python escribe una línea con formato AOF y la deja en el buffer: el dato es visible para otros lectores pero no es durable ante una caída. Después flush y os.fsync lo bajan a disco, que equivale a appendfsync always para ese comando. En la instancia real, redis-cli confirma que appendonly está en yes y appendfsync en everysec, el valor por defecto: un fsync por segundo, con una pérdida máxima de alrededor de un segundo ante una caída.

appendfsync	Qué hace el kernel	Riesgo / costo
no	Deja todo en el page cache; el SO decide cuándo escribir	Muy rápido; ante una caída se pierde hasta ~30 s
everysec (el de mi Redis)	Un fsync por segundo sobre el AOF	Equilibrio: se pierde ≤ 1 s
always	Un fsync por cada escritura	Máxima durabilidad, mínimo rendimiento

Paso 6. Dos errores que se respetan

6b. EMFILE: el límite de descriptores

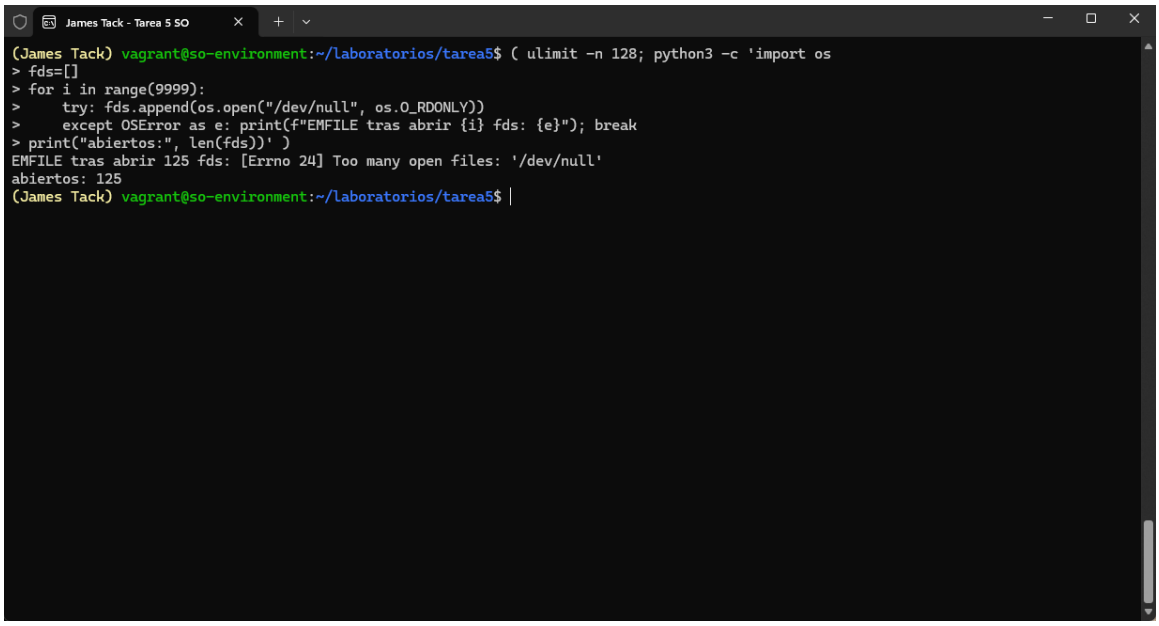


Figura 9. Paso 6b: con ulimit -n 128, Python falla en la apertura 126 con Errno 24, Too many open files.

Dentro de un subshell con ulimit -n 128, Python abre /dev/null en un bucle. Falló después de abrir 125 descriptores: 125 más los 3 estándar suman los 128 del límite. El error es Errno 24, "Too many open files", que es EMFILE. Con un Redis real habría pasado lo mismo cuando los clientes superan el límite del proceso: "maxclients exceeded" y conexiones rechazadas aunque sobre CPU.

6a. ENOSPC por inodos (root, opcional)

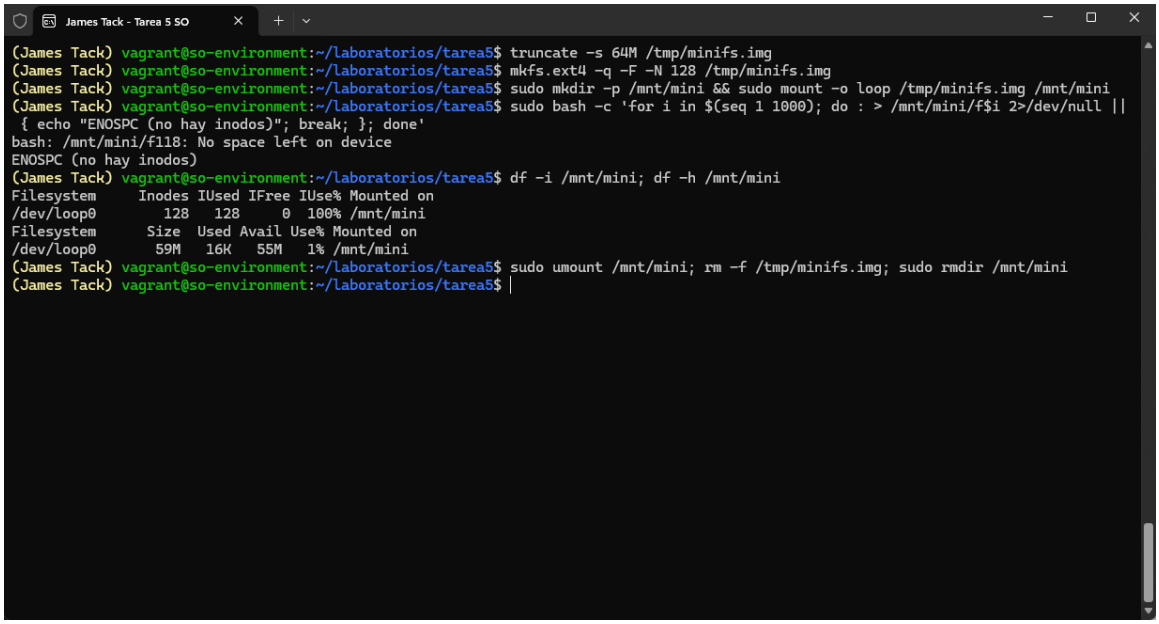


Figura 10. Paso 6a: en un ext4 de 64 MB con 128 inodos, el archivo f118 falla con ENOSPC y df -i marca 100 % de uso.

Con sudo creé una imagen de 64 MB, la formateé en ext4 con solo 128 inodos (mkfs.ext4 -N 128), la monté en loop y creé archivos vacíos hasta que fallara. Falló en f118 con "No space left on device": ext4 reserva inodos propios (11 en este caso), por eso caben 117 archivos y no 128. df -i marca 128 de 128 inodos usados (100 %), mientras que df -h marca solo 16K usados de 59M (1 %). El disco no está lleno de bytes: se agotaron los inodos. Después lo desmonté y borré la imagen.

Tabla de cierre

Lo que vi	Comando que lo demostró	En una frase (pensando en Redis)
Memoria prestada al dataset	free -h; grep -E "^(MemTotal|MemFree|MemAvailable|Cached):" /proc/meminfo	buff/cache (600M) no es memoria perdida: es page cache que el kernel presta a dump.rdb y al AOF y devuelve cuando se necesita, por eso MemAvailable (1.7G) supera a MemFree (1.2G).
Inodos de dump.rdb / appendonly.aof	stat; ls -i; cp + ls -i; df -h /tmp; df -i /tmp	Cada archivo tiene su propio inodo (1034953 y 1034954) y una copia recibe otro; el espacio (18 %) y los inodos (4 %) se agotan por separado, así que un AOF que crece puede dar ENOSPC aunque sobre RAM.
Page cache sobre un archivo tipo RDB	dd + posix_fadvise DONTNEED + time dd (dos veces); grep Cached /proc/meminfo	La 1ª lectura tardó 0.452 s y la 2ª 0.040 s: con el archivo en el page cache, Redis recarga el RDB y reescribe el AOF mucho más rápido.
FDs del servicio vs maxclients / LimitNOFILE	ls -l /proc/$SVC/fd; grep "open files" /proc/$SVC/limits; ulimit -n; redis-cli info clients	Redis tiene 9 FDs (sockets, eventpoll y el AOF) y un límite de 10032 = maxclients (10000) + 32; cada cliente es un socket y gasta un FD.

appendfsync no / everysec / always	demo Python con flush + os.fsync; redis-cli config get appendfsync	Sin fsync el dato está en el page cache y no sobrevive a una caída; Redis usa everysec (pierde ≤ 1 s) y always es lo más seguro pero lo más lento.
ENOSPC vs EMFILE (aplicados a Redis)	ulimit -n 128 + bucle de open(); mkfs.ext4 -N 128 + loop + bucle de creación; df -i	EMFILE (errno 24) es del proceso: demasiados FDs o clientes; ENOSPC es del sistema de archivos: se acabaron los bytes o los inodos, y df -i muestra 100 % con espacio libre.

Conclusiones

Con Redis como ejemplo, las tres ideas del módulo quedan visibles en una hora. Un descriptor es una entrada de la tabla del proceso que puede apuntar a un socket, a un pipe o al archivo AOF, y su techo lo pone RLIMIT_NOFILE. El page cache hace que la segunda lectura de un archivo sea unas once veces más rápida y que Redis recargue su RDB más rápido. Y el fsync es lo único que separa un dato que está en RAM de uno que está en disco, que es la base de appendfsync.

Los dos errores se parecen pero no son iguales. EMFILE es un límite del proceso (en mi prueba, 128 descriptores) y ENOSPC es un límite del sistema de archivos que puede ser de bytes o de inodos. Ver en df -i el 100 % con el espacio casi vacío fue lo más útil, porque es el tipo de diagnóstico que se necesita en el LAB 1 y en la misión final.

Anexos

Anexo A. Scripts de la guía

redis-lab0.sh

```
#!/bin/bash
# LAB 0 — prepara entorno Redis (real si hay binario, simulado si no). Rootless, sin sudo.
set -u
DIR=/tmp/redis-lab0
mkdir -p "$DIR"
USAGE="Uso: $0 {setup|clean|info}"

cmd=${1:-setup}
case "$cmd" in
  setup)
    if command -v redis-server >/dev/null 2>&1; then
      if ! pgrep -x redis-server >/dev/null 2>&1; then
        redis-server --port 7777 --dir "$DIR" --dbfilename dump.rdb \
          --appendonly yes --appendfilename appendonly.aof \
          --daemonize yes --logfile "$DIR/redis.log" 2>/dev/null \
        && echo "redis-server efímero en puerto 7777 (dir=$DIR)" \
        || echo "no se pudo lanzar redis-server; se usa simulación con archivos"
      else
        echo "redis-server ya corría: $(pgrep -o redis-server)"
      fi
```

```
    else
      echo "(sin redis-server instalado: simulación con archivos)"
    fi
    # En ambos casos deja archivos para pasos 3-5
    [ -f "$DIR/dump.rdb" ] || head -c 1048576 /dev/urandom > "$DIR/dump.rdb" 2>/dev/null
    [ -f "$DIR/appendonly.aof" ] || printf '*2\r\n$3\r\nSET\r\n$1\r\nx\r\n' >
"$DIR/appendonly.aof"
    echo "demo" > "$DIR/demo.txt"
    ls -l "$DIR"
    ;;
  info)
    SVC=$(pgrep -o redis-server 2>/dev/null || echo $$)
    echo "PID=$SVC"
    ls -l /proc/$SVC/fd/ 2>/dev/null | head -15
    grep "open files" /proc/$SVC/limits 2>/dev/null
    redis-cli -p 7777 info persistence 2>/dev/null | grep -E "rdb_|aof_" || echo "(sin
redis-cli o sin instancia: ver archivos en $DIR)"
    ;;
  clean)
    redis-cli -p 7777 shutdown nosave 2>/dev/null || true
    rm -rf "$DIR"
    echo "limpio"
    ;;
  *) echo "$USAGE" ;;
esac
```

baseline.sh

```
#!/bin/bash
# LAB 0 — baseline: kernel + servicio + page cache (sin sudo, con o sin Redis)
set -u
# Redis del lab (7777) o tu shell. NO usar sshd/nginx: /proc/<pid>/fd daría Permission
denied.
SVC=$(pgrep -of '[r]edis-server.*7777' 2>/dev/null || echo $$)
echo "PID_SERVICIO=$SVC"
tr '\0' ' ' < /proc/$SVC/cmdline 2>/dev/null; echo
grep -E "Name|PPid|Threads|VmRSS|State" /proc/$SVC/status 2>/dev/null
uname -r; cat /proc/version; uptime -p
echo "--- mem ---"
free -h
grep -E '^(MemTotal|MemFree|MemAvailable|Cached|Dirty|Writeback):' /proc/meminfo
echo "--- fds servicio ---"
ls -l /proc/$SVC/fd/ 2>/dev/null | head -15
grep "open files" /proc/$SVC/limits 2>/dev/null
ulimit -n
```